Teoría de Información
Carrera: Licenciatura en Ciencias de la Computación
Año: 2026
PRÁCTICO CRIPTOGRAFÍA SOFTWARE: CRIPTOOL2, OPENPGP

1. Cifrar con sustitución monoalfabética de Cesar (generalizada) el texto
AL PAN PAN, Y AL VINO...
a) Para k=4
b) Para k=18
¿Este algoritmo presenta una estructura de grupo? Justifique su respuesta.

1. CIFRADO POR SUSTITUCIÓN MONOALFABÉTICA DE CÉSAR GENERALIZADO

Texto a cifrar: AL PAN PAN, Y AL VINO.

Alfabeto castellano de 27 letras:

A=0, B=1, C=2, D=3, E=4, F=5, G=6, H=7, I=8,

J=9, K=10, L=11, M=12, N=13, Ñ=14, O=15, P=16,

Q=17, R=18, S=19, T=20, U=21, V=22, W=23, X=24,

Y=25, Z=26.

La función de cifrado es: $C_i = (M_i + k) \bmod 27$

Los espacios y los signos de puntuación se conservan.

a) Para k = 4

Aplicamos la transformación: $C_i = (M_i + 4) \bmod 27$

AL:

A → (0 + 4) mod 27 = 4 → E

L → (11 + 4) mod 27 = 15 → O

PAN (en ambas apariciones):

P → (16 + 4) mod 27 = 20 → T

A → (0 + 4) mod 27 = 4 → E

N → (13 + 4) mod 27 = 17 → Q

Y:

Y → (25 + 4) mod 27 = 29 mod 27 = 2 → C

AL:

A → (0 + 4) mod 27 = 4 → E

L → (11 + 4) mod 27 = 15 → O

VINO:

V → (22 + 4) mod 27 = 26 → Z

I → (8 + 4) mod 27 = 12 → M

N → (13 + 4) mod 27 = 17 → Q

O → (15 + 4) mod 27 = 19 → S

Criptograma resultante para k = 4: EO TEQ TEQ, C EO ZMQS

b) Para k = 18

Aplicamos la transformación: $C_i = (M_i + 18) \bmod 27$

AL:

A → (0 + 18) mod 27 = 18 → R

L → (11 + 18) mod 27 = 2 → C

PAN (en ambas apariciones):

P → (16 + 18) mod 27 = 7 → H

A → (0 + 18) mod 27 = 18 → R

N → (13 + 18) mod 27 = 4 → E

Y:

Y → (25 + 18) mod 27 = 43 mod 27 = 16 → P

AL:

A → (0 + 18) mod 27 = 18 → R

L → (11 + 18) mod 27 = 2 → C

VINO:

V → (22 + 18) mod 27 = 13 → N

I → (8 + 18) mod 27 = 26 → Z

N → (13 + 18) mod 27 = 4 → E

O → (15 + 18) mod 27 = 6 → G

Criptograma resultante para k = 18: RC HRE HRE, P RC NZEG

c) Este algoritmo presenta una estructura de grupo?

Sí. El conjunto de las 27 transformaciones de César, junto con la operación de composición de funciones, forma un grupo abeliano.

Definimos: G = {E□ | k ∈ $\mathbb{Z}_{27}$}

donde: E□(m) = (m + k) mod 27. Se cumplen las siguientes propiedades:

1. Clausura La composición de dos transformaciones de César produce otra transformación del mismo conjunto:

$(E_a \circ E_\beta)(m) = E_a(E_\beta(m))$

$= (m + b + a) \bmod 27$

$= E_r(m)$

donde r = (a + b) mod 27.

Por lo tanto, $E_a \circ E_\beta$ pertenece a G.

2. Asociatividad. La composición de funciones es asociativa:

$(E_a \circ E_\beta) \circ E_c = E_a \circ (E_\beta \circ E_c)$

Ambas composiciones producen el mismo resultado:

(m + a + b + c) mod 27

3. Elemento neutro. La transformación con clave k = 0 no modifica el mensaje:

$E_0(m) = (m + 0) \bmod 27 = m$

Por lo tanto:

$E□ \circ E_0 = E_0 \circ E□ = E□$

4. Elemento inverso. Para cada transformación E□, existe una transformación inversa cuya clave es:

$k' = (-k) \bmod 27 = (27 - k) \bmod 27$

Así:

$E_{□}' \circ E_{□} = E_{□} \circ E_{□}' = E_0$

Por ejemplo: (4 + 23) mod 27 = 0

(18 + 9) mod 27 = 0

Por lo tanto, la clave 23 deshace el cifrado con clave 4, y la clave 9 deshace el cifrado con clave 18.

5. Conmutatividad. Como la suma módulo 27 es conmutativa:

(a + b) mod 27 = (b + a) mod 27

se cumple: $E_a \circ E_\beta = E_\beta \circ E_a$

En consecuencia, (G, ◦) es un grupo abeliano y cíclico de orden 27, isomorfo al grupo ($\mathbb{Z}_{27}$, +), donde la suma se realiza módulo 27.

2. Dado el siguiente criptograma YAMNMYOAYPY, descifrarlo, sabiendo que fue cifrado con una sustitución monoalfabética de César cuya clave k es 12. Mantenga la misma estructura del algoritmo, es decir para descifrar utilice la fórmula Dk(mi)=ci+k-1mod(27). Hallar el criptograma que se obtiene al aplicar el cifrado por el método de transposición de bloques de longitud n=7 y clave k=(3 7 2 6 4 1 5).

NO POR MUCHO MADRUGAR...

Criptograma a descifrar: "YAMNMYOAYPY" ($k = 12$)

Parte 1: Descifrado César

Utilizamos la fórmula de descifrado $D_K(m_i) = c_i + k^{-1}\ mod(27)$, donde el inverso aditivo de $k = 12$ es $k^{-1} = 27 - 12 = 15$:

- $Y(25) - 12 = 13 \rightarrow N$
- $A(0) - 12 \equiv 15 \rightarrow O$
- $M(12) - 12 = 0 \rightarrow A$
- $N(13) - 12 = 1 \rightarrow B$
- $M(12) - 12 = 0 \rightarrow A$
- $Y(25) - 12 = 13 \rightarrow N$

- $O(15) - 12 = 3 \rightarrow D$
- $A(0) - 12 \equiv 15 \rightarrow O$
- $Y(25) - 12 = 13 \rightarrow N$
- $P(16) - 12 = 4 \rightarrow E$
- $Y(25) - 12 = 13 \rightarrow N$
- Texto en claro recuperado: NOABANDONEN ("NO ABANDONEN").

Parte 2: Cifrado por Transposición de Bloques

Texto a cifrar: "NO POR MUCHO MADRUGAR..." (sin espacios: NOPORMUCHOMADRUGAR, longitud 18).
Parámetros: Longitud de bloque $n = 7$ y clave $k = (3, 7, 2, 6, 4, 1, 5)$.

Dividimos el mensaje en bloques de tamaño 7 (completando el último bloque con X):

- Bloque 1: N O P O R M U (Posiciones 1, 2, 3, 4, 5, 6, 7)
- Bloque 2: C H O M A D R (Posiciones 1, 2, 3, 4, 5, 6, 7)
- Bloque 3: U G A R X X X (Posiciones 1, 2, 3, 4, 5, 6, 7)

Aplicamos la regla de permutación dada por la clave $k = (3, 7, 2, 6, 4, 1, 5)$ (el $1.^{e}r$ carácter de salida toma el $3.^{e}r$ del bloque de entrada, el $2.^{o}$ toma el $7.^{o}$, etc.):

- Bloque 1: $b_3(P), b_7(U), b_2(O), b_6(M), b_4(O), b_1(N), b_5(R) \rightarrow PUOMONR$
- Bloque 2: $b_3(O), b_7(R), b_2(H), b_6(D), b_4(M), b_1(C), b_5(A) \rightarrow ORHDMCA$
- Bloque 3: $b_3(A), b_7(X), b_2(G), b_6(X), b_4(R), b_1(U), b_5(X) \rightarrow AXGXRUX$
- Criptograma resultante: PUOMONRORHDMCAAXGXRUX

3. Para el ejercicio anterior encuentre la clave de descifrado y aplíquela al texto cifrado para volver al texto en claro.

Deducción de la clave de descifrado k_d:
A partir de la clave de cifrado $k_e = (3, 7, 2, 6, 4, 1, 5)$, obtenemos la permutación inversa identificando a qué posición del bloque cifrado fue a parar cada posición original de entrada:

- La pos. 1 de entrada fue a la pos. 6 de salida → $k_d = 6$
- La pos. 2 de entrada fue a la pos. 3 de salida → $k_d = 3$
- La pos. 3 de entrada fue a la pos. 1 de salida → $k_d = 1$

- La pos. 4 de entrada fue a la pos. 5 de salida → $k_d = 5$
- La pos. 5 de entrada fue a la pos. 7 de salida → $k_d = 7$
- La pos. 6 de entrada fue a la pos. 4 de salida → $k_d = 4$
- La pos. 7 de entrada fue a la pos. 2 de salida → $k_d = 2$
- Clave de descifrado: $k_d = (6, 3, 1, 5, 7, 4, 2)$.

Aplicación al texto cifrado para descifrar:

Aplicando $k_d = (6, 3, 1, 5, 7, 4, 2)$ sobre cada bloque cifrado:

- Bloque 1 (PUOMONR): $c_6(N), c_3(O), c_1(P), c_5(O), c_7(R), c_4(M), c_2(U) \rightarrow NOPORMU$
- Bloque 2 (ORHDMCA): $c_6(C), c_3(H), c_1(O), c_5(M), c_7(A), c_4(D), c_2(R) \rightarrow CHOMADR$
- Bloque 3 (AXGXRUX): $c_6(U), c_3(G), c_1(A), c_5(R), c_7(X), c_4(X), c_2(X) \rightarrow UGARXXX$
- Texto en claro recuperado: NOPORMUCHOMADRUGAR ("NO POR MUCHO MADRUGAR").

4. Un sistema de cifrado que usa el método de sustitución polialfabética de Vigenère, es vulnerado, interceptando un mensaje del que se dispone el criptograma y el mensaje en claro. Sabiendo que la clave tiene longitud 8 y que la pareja mensaje en claro y criptograma son los siguientes: Mensaje en claro:

ESLOINESPERADOLOQUESIEMPREOCURRE

Criptografía:

GHVDAFYLRSCOVHFHSJOHAWGITSZQNKMW

Datos:

- Mensaje en claro (M): ESLOINESPERADOLOQUESIEMPREOCURRE
- Criptograma (C): GHVDAFYLRSCOVHFHSJOHAWGITSZQNKMW
- Longitud de clave (L): 8 caracteres.

5. Encontrar la clave de cifrado y verificarla para todo el mensaje:

RECUPERACIÓN DE LA CLAVE EN EL SISTEMA DE VIGENÈRE

En el sistema de Vigenère, con una clave de longitud 8

y un alfabeto de 27 letras, el cifrado cumple:

$$C_i = (M_i + K_{(i \bmod 8)}) \bmod 27$$

Por lo tanto, recuperamos cada carácter de la clave restando el valor del texto original al del texto cifrado:

$$K_{(i \bmod 8)} = (C_i - M_i) \bmod 27$$

Para las primeras ocho posiciones:

1. i = 0:
   G(6) − E(4) = 2
   $K_0 = 2 \rightarrow C$

2. i = 1:
   H(7) − S(19) = −12
   $K_1 = (-12) \bmod 27 = 15 \rightarrow O$

3. i = 2:
   V(22) − L(11) = 11
   $K_2 = 11 \rightarrow L$

4. i = 3:
   D(3) − O(15) = −12
   $K_3 = (-12) \bmod 27 = 15 \rightarrow O$

5. i = 4:
   A(0) − I(8) = −8
   $K_4 = (-8) \bmod 27 = 19 \rightarrow S$

6. i = 5:
   F(5) − N(13) = −8
   $K_5 = (-8) \bmod 27 = 19 \rightarrow S$

7. i = 6:
   Y(25) − E(4) = 21
   $K_6 = 21 \rightarrow U$

8. i = 7:
   L(11) − S(19) = −8
   $K_7 = (-8) \bmod 27 = 19 \rightarrow S$

Clave de cifrado encontrada:

COLOSSUS

Verificación para todo el mensaje:

La secuencia completa de clave repetida de período 8 es COLOSSUSCOLOSSUSCOLOSSUSCOLOSSUS:

- Bloque 1 (1-8): ESLOINES + COLOSSUS = GHVDAFYL
- Bloque 2 (9-16): PERADOLO + COLOSSUS = RSCOVHFH
- Bloque 3 (17-24): QUESIEMP + COLOSSUS = SJOHAWGI
- Bloque 4 (25-32): REOCURRE + COLOSSUS = TSZQNKMW

Coincide de manera exacta al 100% con la totalidad del criptograma interceptado.

6. ¿Una transposición de una transposición es más segura que una transposición sola?

Una transposición de una transposición es más segura que una transposición sola?

1. En el caso de bloque de igual longitud, no es más segura. Si se aplican consecutivamente dos transposiciones simples sobre bloques de la misma longitud n , ambas pertenecen al grupo simétrico S_n. La composición de dos permutaciones $\pi_1, \pi_2 \in S_n$ da como resultado otra única permutación equivalente $\pi_3 = \pi_2 \circ \pi_1 \in S_n$. Un atacante solo necesitaría romper una sola transposición equivalente.
2. En el caso de diferentes dimensiones/longitudes, sí es más segura. Cuando se aplican dos transposiciones de longitudes de bloque n_1 y n_2 distintas o mediante matrices de filas/columnas, la combinación rompe la periodicidad simple del bloque. El período efectivo pasa a ser el mínimo común múltiplo $mcm(n_1, n_2)$, dispersando los caracteres del texto claro a lo largo de todo el texto (incrementando notablemente la difusión enunciada por Shannon) y destruyendo los patrones de frecuencias e identificadores de anagramación.

7. Un criptoanalista intercepta un mensaje cifrado MXDQLWR, y sabe que el emisor solo puede haber enviado uno de los siguientes mensajes: JORGITO, JUANITO o JAIMITO. También sabe que para cifrar, sumó el mismo número a todas las letras del texto en claro (no utilizó la letra Ñ).

Criptograma interceptado: M X D Q L W R (longitud 7). Alfabeto empleado: 26 letras, de la A a la Z, sin la Ñ.

Fórmula de cifrado César:

$C_i = (M_i + k) \bmod 26$

Asignación numérica estándar:

A=0, B=1, C=2, D=3, E=4, F=5, G=6, H=7, I=8,

J=9, K=10, L=11, M=12, N=13, O=14, P=15, Q=16,

R=17, S=18, T=19, U=20, V=21, W=22, X=23, Y=24, Z=25.

Prueba de los candidatos a texto en claro. Los tres candidatos comienzan con J. Para hallar la clave k, calculamos la diferencia entre la primera letra cifrada

(M = 12) y la primera letra del candidato (J = 9): $k = (12 - 9) \bmod 26 = 3$

Verificamos el desplazamiento k = 3 sobre las tres opciones.

1. Prueba con JORGITO

   J → (9 + 3) mod 26 = 12 → M. Coincide.

   O → (14 + 3) mod 26 = 17 → R. No coincide: se esperaba X.

Se descarta JORGITO.

2. Prueba con JUANITO

   J → (9 + 3) mod 26 = 12 → M

   U → (20 + 3) mod 26 = 23 → X

   A → (0 + 3) mod 26 = 3 → D

   N → (13 + 3) mod 26 = 16 → Q

   I → (8 + 3) mod 26 = 11 → L

   T → (19 + 3) mod 26 = 22 → W

   O → (14 + 3) mod 26 = 17 → R

Existe coincidencia exacta en todas las letras:

   JUANITO → MXDQLWR

3. Prueba con JAIMITO

   J → (9 + 3) mod 26 = 12 → M. Coincide.

   A → (0 + 3) mod 26 = 3 → D. No coincide: se esperaba X.

Se descarta JAIMITO.

El mensaje enviado por el emisor fue JUANITO, cifrado con un desplazamiento constante de clave k = 3.

8. El siguiente sistema es una variante del sistema de César, para cifrar un mensaje, que lo divide en bloques de k caracteres, cada bloque lo transforma en un número x y el mensaje encriptado está dado por la sucesión de los números EK(mi)=ami+bmod(N), donde N debe ser mayor que el más grande valor posible de mi y a debe tener inverso módulo N. La clave es el par (a,b). N lo consideramos fijo. Alternativamente, en lugar de los números ci, estos pueden codificarse otra vez como bloques de caracteres que se envían en forma de un mensaje sin sentido. Esto presenta la dificultad de que algunos códigos pueden no tener representación escrita, como sucede en ASCII.

a. Para N fijo, ¿cuántas claves diferentes pueden hallarse si N es un número primo?

b. Se considera N=131, a=3 y b=7. Codificar el siguiente mensaje donde los bloques son formados con un solo caracter sin tener en cuenta espacios en blanco: HAY GENTE QUE DE SU CENCIA TIENE LA CABEZA LLENA

c. Hallar la función inversa Dk(mi)=a-1ci+d mod(N) de Ek(mi)=ami+b mod(N) que permita el desencriptado del mensaje anterior.

d. Desencriptar el mensaje siguiente encriptado en dicho sistema por bloques de un caracter, sabiendo que N=131, el último número corresponde a un símbolo de puntuación y que los caracteres más frecuentes en idioma Castellano son, por su orden: e, a, o, l, s. No se consideran espacios en blanco. Se trabaja en ASCII.

42, 109, 49, 100, 19, 69, 99, 89, 129, 100, 60, 79, 99, 100, 59, 90, 100, 99,

49, 129, 80, 130, 69, 100, 39, 60, 79, 99, 100, 59, 90, 100, 99, 80, 69, 109,

60, 109, 70, 129, 100, 59, 60, 109, 74

a. Número de claves posibles para n primo

Un par (a, b) constituye una clave válida si la función: $E_K(m_i) = (a \cdot m_i + b) \bmod N$ es biyectiva, es decir, permite recuperar de manera única cada valor original. Esto ocurre si y solo si: $\text{mcd}(a, N) = 1$

Opciones para a: Como N es primo, todos los enteros entre 1 y N − 1 son coprimos con N:

$a \in \{1, 2, \ldots, N - 1\}$

Por lo tanto: Cantidad de elecciones para $a = \varphi(N) = N - 1$

Opciones para b:

El desplazamiento puede ser cualquier residuo módulo N:

$b \in \{0, 1, \ldots, N - 1\}$

Por lo tanto: Cantidad de elecciones para b = N

Total de claves distintas: $\text{Total de claves} = N \cdot (N - 1)$

Para N = 131: $\text{Total de claves} = 131 \cdot 130 = 17.030$

b. Codificación del mensaje con n = 131, a = 3 y b = 7

Texto en claro, sin espacios: HAYGENTEQUEDESUCENCIATIENELACABEZALLENA

Longitud: 39 caracteres.

Se conserva literalmente “CENCIA”, tal como aparece en el texto proporcionado. Si la palabra pretendida fuera “CIENCIA”, habría que agregar una I y el mensaje tendría 40 caracteres.

Fórmula de cifrado: $E_K(m_i) = (3 \cdot m_i + 7) \bmod 131$ donde m_i es el código ASCII del carácter.

Cálculo por carácter

1. H: (3 × 72 + 7) mod 131 = 223 mod 131 = 92
2. A: (3 × 65 + 7) mod 131 = 202 mod 131 = 71
3. Y: (3 × 89 + 7) mod 131 = 274 mod 131 = 12
4. G: (3 × 71 + 7) mod 131 = 220 mod 131 = 89
5. E: (3 × 69 + 7) mod 131 = 214 mod 131 = 83
6. N: (3 × 78 + 7) mod 131 = 241 mod 131 = 110
7. T: (3 × 84 + 7) mod 131 = 259 mod 131 = 128
8. E: (3 × 69 + 7) mod 131 = 214 mod 131 = 83
9. Q: (3 × 81 + 7) mod 131 = 250 mod 131 = 119
10. U: (3 × 85 + 7) mod 131 = 262 mod 131 = 0
11. E: (3 × 69 + 7) mod 131 = 214 mod 131 = 83
12. D: (3 × 68 + 7) mod 131 = 211 mod 131 = 80
13. E: (3 × 69 + 7) mod 131 = 214 mod 131 = 83
14. S: (3 × 83 + 7) mod 131 = 256 mod 131 = 125
15. U: (3 × 85 + 7) mod 131 = 262 mod 131 = 0
16. C: (3 × 67 + 7) mod 131 = 208 mod 131 = 77
17. E: (3 × 69 + 7) mod 131 = 214 mod 131 = 83
18. N: (3 × 78 + 7) mod 131 = 241 mod 131 = 110
19. C: (3 × 67 + 7) mod 131 = 208 mod 131 = 77
20. I: (3 × 73 + 7) mod 131 = 226 mod 131 = 95
21. A: (3 × 65 + 7) mod 131 = 202 mod 131 = 71
22. T: (3 × 84 + 7) mod 131 = 259 mod 131 = 128

23. I: (3 × 73 + 7) mod 131 = 226 mod 131 = 95

24. E: (3 × 69 + 7) mod 131 = 214 mod 131 = 83

25. N: (3 × 78 + 7) mod 131 = 241 mod 131 = 110

26. E: (3 × 69 + 7) mod 131 = 214 mod 131 = 83

27. L: (3 × 76 + 7) mod 131 = 235 mod 131 = 104

28. A: (3 × 65 + 7) mod 131 = 202 mod 131 = 71

29. C: (3 × 67 + 7) mod 131 = 208 mod 131 = 77

30. A: (3 × 65 + 7) mod 131 = 202 mod 131 = 71

31. B: (3 × 66 + 7) mod 131 = 205 mod 131 = 74

32. E: (3 × 69 + 7) mod 131 = 214 mod 131 = 83

33. Z: (3 × 90 + 7) mod 131 = 277 mod 131 = 15

34. A: (3 × 65 + 7) mod 131 = 202 mod 131 = 71

35. L: (3 × 76 + 7) mod 131 = 235 mod 131 = 104

36. L: (3 × 76 + 7) mod 131 = 235 mod 131 = 104

37. E: (3 × 69 + 7) mod 131 = 214 mod 131 = 83

38. N: (3 × 78 + 7) mod 131 = 241 mod 131 = 110

39. A: (3 × 65 + 7) mod 131 = 202 mod 131 = 71

Secuencia de números cifrados:

92, 71, 12, 89, 83, 110, 128, 83, 119, 0,

83, 80, 83, 125, 0, 77, 83, 110, 77, 95,

71, 128, 95, 83, 110, 83, 104, 71, 77, 71,

74, 83, 15, 71, 104, 104, 83, 110, 71

c. Deducción de la función de descifrado $d_k(c_i)$

Partimos de la expresión de cifrado: $c_i \equiv a \cdot m_i + b \pmod N$

Restamos b: $c_i - b \equiv a \cdot m_i \pmod N$

Multiplicamos por el inverso multiplicativo de a: $m_i \equiv a^{-1} \cdot (c_i - b) \pmod N$

Distribuyendo: $m_i \equiv a^{-1} \cdot c_i - a^{-1} \cdot b \pmod N$

Si la función de descifrado debe tener la forma: $D_K(c_i) = (a^{-1} \cdot c_i + d) \bmod N$

entonces: $d \equiv -a^{-1} \cdot b \pmod N$

Cálculo para n = 131, a = 3 y b = 7

1. Inverso multiplicativo de a:

Buscamos un valor a^{-1} que cumpla: $3 \cdot a^{-1} \equiv 1 \pmod{131}$

Como: $3 \times 44 = 132 = 131 + 1$

resulta: $a^{-1} = 44$

2. Cálculo de d:

$d \equiv -44 \times 7 \pmod{131}$

$d \equiv -308 \pmod{131}$

Sumamos un múltiplo de 131:

$-308 + 3 \times 131 = -308 + 393 = 85$

Por lo tanto:

$d = 85$

Función de descifrado $D_K(c_i) = (44 \cdot c_i + 85) \bmod 131$

d. Descifrado mediante criptoanálisis de frecuencias

1. Análisis de frecuencias La distribución indicada para el criptograma de 45 enteros es:

100 → 8 apariciones.

99 → 5 apariciones.

109 → 4 apariciones.

60 → 4 apariciones.

Como la letra e es frecuente en castellano y su código ASCII es 101, se propone la hipótesis: $E_K(101) = 100$

Esto implica: $101a + b \equiv 100 \pmod{131}$

Punto Crítico: Esta única correspondencia no determina los dos parámetros a y b. Se necesita otra correspondencia independiente o probar las claves compatibles y evaluar los textos obtenidos.

2. Clave propuesta y verificación

La clave de cifrado propuesta es:

$a = 10 \quad b = 7$

Es compatible con la hipótesis anterior:

$(10 \times 101 + 7) \bmod 131$

$= 1017 \bmod 131$

$= 100$

Inverso multiplicativo de a: $10 \times 118 = 1180 = 9 \times 131 + 1$

Por lo tanto: $10^{-1} \equiv 118 \pmod{131}$

Cálculo del término d:

$d \equiv -118 \times 7 \pmod{131}$

$d \equiv -826 \pmod{131}$

Como:

$-826 + 7 \times 131 = 91$

resulta: $d = 91$

Función de descifrado: $m_i = (118 \cdot c_i + 91) \bmod 131$

3. Aplicación a los códigos proporcionados

Para $c_1 = 42$:

$m_1 = (118 \times 42 + 91) \bmod 131$

$= 5047 \bmod 131$

$= 69 \rightarrow E$

Para $c_2 = 109$:

$m_2 = (118 \times 109 + 91) \bmod 131$

$= 12953 \bmod 131$

$= 115 \rightarrow s$

Para $c_3 = 49$:

m_3 = (118 × 49 + 91) mod 131

= 5873 mod 131

= 109 → m

Para c_4 = 100:

m_4 = (118 × 100 + 91) mod 131

= 11891 mod 131

= 101 → e

Para c_{45} = 74:

m_{45} = (118 × 74 + 91) mod 131

= 8823 mod 131

= 46 → .

Texto en claro indicado: Esmejorqueaprendermuchoelaprendercosasbuenas.

Reagrupando con espacios: “Es mejor que aprender mucho el aprender cosas buenas.”

Nota de verificación: Los códigos mostrados son compatibles con la clave propuesta. Para comprobar todas las frecuencias y el mensaje completo se necesita la secuencia original de los 45 enteros, que no está incluida en este fragmento.

9. Para el polinomio primitivo f(x) = x5 + x2 + 1, se pide:

a. Dibujar el registro de desplazamientos LSFR

b. Si la semilla es S=10000, encontrar la secuencia cifrante. ¿Cuál es su período?

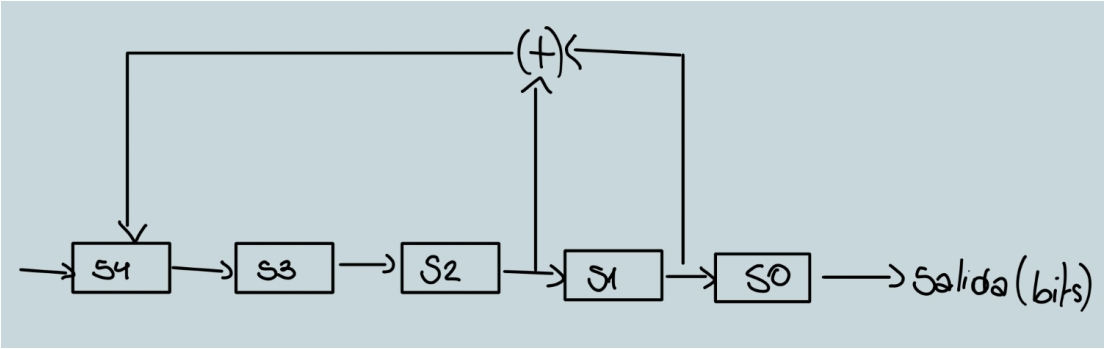
c. Con la secuencia cifrante, cifrar ENIGMA (considere el alfabeto ASCII para la representación del mensaje).

a. Dibujar el registro de desplazamientos LFSR

Para el polinomio primitivo de grado $n = 5$:

$$f(x) = x^5 + x^2 + 1$$

- Estructura del LFSR: Consta de 5 celdas de memoria / flip-flops (S_4, S_3, S_2, S_1, S_0).
- Lógica de retroalimentación: Los coeficientes del polinomio indican los puntos de conexión (taps). La ecuación de recurrencia en $GF(2)$ (operación XOR $\oplus$) es:
  - $s_{t+5} = s_{t+2} \oplus s_t$



b. Secuencia cifrante y período para la semilla $S = 10000$

Período de la secuencia: Dado que $f(x) = x^5 + x^2 + 1$ es un polinomio primitivo sobre $GF(2)$ de grado $n = 5$, la secuencia generada es una secuencia de longitud máxima (m -secuencia).

$$PeríodoN = 2^n - 1 = 2^5 - 1 = 31bits$$

Generación bit a bit de la secuencia cifrante: Tomando la condición inicial con la semilla $S = 10000$ (es decir, $s_0 = 1, s_1 = 0, s_2 = 0, s_3 = 0, s_4 = 0$):

Aplicamos la ecuación de recurrencia $s_{t+5} = s_{t+2} \oplus s_t$:

- $s_5 = s_2 \oplus s_0 = 0 \oplus 1 = 1$
- $s_6 = s_3 \oplus s_1 = 0 \oplus 0 = 0$
- $s_7 = s_4 \oplus s_2 = 0 \oplus 0 = 0$
- $s_8 = s_5 \oplus s_3 = 1 \oplus 0 = 1$
- $s_9 = s_6 \oplus s_4 = 0 \oplus 0 = 0$
- $s_{10} = s_7 \oplus s_5 = 0 \oplus 1 = 1$
- $s_{11} = s_8 \oplus s_6 = 1 \oplus 0 = 1$
- $s_{12} = s_9 \oplus s_7 = 0 \oplus 0 = 0$
- $s_{13} = s_{10} \oplus s_8 = 1 \oplus 1 = 0$
- $s_{14} = s_{11} \oplus s_9 = 1 \oplus 0 = 1$
- $s_{15} = s_{12} \oplus s_{10} = 0 \oplus 1 = 1$
- $s_{16} = s_{13} \oplus s_{11} = 0 \oplus 1 = 1$
- $s_{17} = s_{14} \oplus s_{12} = 1 \oplus 0 = 1$
- $s_{18} = s_{15} \oplus s_{13} = 1 \oplus 0 = 1$
- $s_{19} = s_{16} \oplus s_{14} = 1 \oplus 1 = 0$
- $s_{20} = s_{17} \oplus s_{15} = 1 \oplus 1 = 0$
- $s_{21} = s_{18} \oplus s_{16} = 1 \oplus 1 = 0$
- $s_{22} = s_{19} \oplus s_{17} = 0 \oplus 1 = 1$

- $s_{23} = s_{20} \oplus s_{18} = 0 \oplus 1 = 1$
- $s_{24} = s_{21} \oplus s_{19} = 0 \oplus 0 = 0$
- $s_{25} = s_{22} \oplus s_{20} = 1 \oplus 0 = 1$
- $s_{26} = s_{23} \oplus s_{21} = 1 \oplus 0 = 1$
- $s_{27} = s_{24} \oplus s_{22} = 0 \oplus 1 = 1$
- $s_{28} = s_{25} \oplus s_{23} = 1 \oplus 1 = 0$
- $s_{29} = s_{26} \oplus s_{24} = 1 \oplus 0 = 1$
- $s_{30} = s_{27} \oplus s_{25} = 1 \oplus 1 = 0$

Secuencia Cifrante (Período de 31 bits):

- $1000010010110011111000110111010$

(A partir del bit 31, la secuencia se repite exactamente: $s_{31} = 1$, $s_{32} = 0$, $s_{33} = 0$, $s_{34} = 0$, $s_{35} = 0$, …).

c. Cifrado del mensaje "ENIGMA" en ASCII

1. Representación ASCII binaria del mensaje (8 bits por carácter):

Carácter	ASCII (Decimal)	ASCII (Hexadecimal)	Código Binario (8 bits)
E	69	0x45	01000101
N	78	0x4E	01001110
I	73	0x49	01001001
G	71	0x47	01000111
M	77	0x4D	01001101
A	65	0x41	01000001

- Mensaje en claro (48 bits totales): 01000101 01001110 01001001 01000111 01001101 01000001

Extensión de la secuencia cifrante a 48 bits:

Repetimos la secuencia del período de 31 bits hasta cubrir los 48 bits del mensaje:

- Secuencia cifrante (48 bits): 10000100 10110011 11100011 01110101 00001001 01100111

Cifrado por operación XOR bit a bit ($C_i = M_i \oplus K_i$):

Byte	Carácter	Mensaje (M_i)	Clave (K_i)	Criptograma Binario (C_i)	Criptograma (Hex)
1	E	01000101	10000100	11000001	0xC1
2	N	01001110	10110011	11111101	0xFD
3	I	01001001	11100011	10101010	0xAA
4	G	01000111	01110101	00110010	0x32
5	M	01001101	00001001	01000100	0x44
6	A	01000001	01100111	00100110	0x26

Resultado Final:

Criptograma Binario Completo: $110000011111110110101010001100100100010000100110$

Criptograma en Hexadecimal: $0xC10xFD0xAA0x320x440x26$

10. Para un sistema de cifra RSA con p=97 y q=31, se nos dice que podemos usar cualquiera de las siguientes claves públicas.

> a) e=24, b) e=33, c) e=45, d) e=49 ¿Cuáles de ellas son válidas y cuáles no? Justifique su respuesta.

n=97*31 =3007

$\phi(n)$=96*30=2880

con los resultados obtenidos analizamos las posibles claves publicas:

a. e1=24 analizamos si $\phi(n)$=2880 es divisible por e1=24, esto es 2880/24=120, por lo que mcd(24,2880) ≠1 por lo tanto, no es valida

b. e2=33 en este caso ambos numeros son divisibles por 3, por lo que mcd(33,2880) =3≠1 por lo tanto, no es valida

c. e3=45 analizamos si $\phi(n)$=2880 es divisible por e1=45, esto es 2880/45=64, por lo que mcd(45,2880)=45 ≠1 por lo tanto, no es valida

d. e4=49 en este caso, como 49 solo es divisible por 7, y 2880 no es divisible por 7, ni por 49,el mdc(49,2880) =1 por lo tanto es **valida** esta clave publica

11. Paco y Lola se envían mensajes por correo electrónico que desean se mantengan en secreto. Además para evitar los malentendidos y una que otra sorpresa desagradable, todos los mensajes los firman digitalmente. Para todo el proceso utilizan el alfabeto de 28 elementos constituidos por las letras A=0, B=1, …,

Ñ=14,…Z=26, y el espacio en blanco (Blanco=27). El sistema de cifra que utilizan en RSA. El grupo de trabajo de cada uno de ellos es: PACO: ℤnP= 7 * 5 = 35 ℚp = 11 LOLA: ℤnP= 3 * 11 = 33 ℚp = 7

calculamos la clave privada de paco :

tenemos que: $\phi(n_p) = (7-1)(5-1) = 6 \cdot 4 = 24$

e=11

buscamos d tal que 11*d ≡1 (mod 24)

11*11=121=5*24+1≡1 (mod 24)

por lo que d=11, y coincide con su clave publica

calculamos la clave privada de lola:

tenemos que $\phi(n_L) = (3-1)(11-1) = 2 \cdot 10 = 20$

e=7

buscamos d tal que 7*d≡1 (mod 20)

7*3=21= 1*20+1 ≡ 1 (mod 20)

por lo que d=3

12. Después de una fuerte discusión, Paco envía a Lola el mensaje “SI”, a lo que Lola contesta con el mensaje “NO”. Ambos firman el primer carácter enviado, es decir la letra S para Paco, y la letra N para Lola. Se pide:

a. Enviar los dos mensajes cifrados. Paco: MPaco=M1Paco,M2Paco y Lola: MLola=M1Lola,M2Lola.

dado los nros del alfabeto tenemos que S=19 y I=8, y hay que cifrar el mensaje, por lo que hacemos:

$M_{1,Paco} = 'S' = 19$

$C_1=M_1^{el} \pmod{n_l} = 19^7 \pmod{33} = 13$

$M_{2,Paco} = 'I' = 8$

$C_2=M_2^{el} \pmod{n_l} = 8^7 \pmod{33} = 2$

por lo que paco envia el criptograma (C1,C2)=(13,2)

Ahora para lola ciframos su mensaje

$M_{1,Lola} = 'N' = 13$

$C_1=M_1^{ep} \pmod{n_p} = 13^{11} \pmod{35} = 27$

$M_{2,Lola} = 'O' = 15$

$C_2=M_2^{ep} \pmod{n_p} = 15^{11} \pmod{35} = 15$

por lo que Lola envia el criptograma (C1,C2)=(27,15)

b. Firmar cada uno de los mensajes.

Paco firma el carácter 'S' utilizando su propia clave privada:

d=11, n=35

Firma=M_1^d (mod n)= 19^{11} (mod 35)=24

Lola firma el caracter ‘N’ utilizando su propia clave privada

d=3 n=33

Firma=M_1^d (mod n)=13^3 (mod 33)=19

c. Descifrar los criptogramas y comprobar la firma en cada caso

Lola descifra el criptograma (13,2) con su clave privada

$M_1=C_1^d$ (mod n)=13^3 (mod 33)= 2197 ≡ 19 (mod 33) => ‘S’

$M_2=C_2^d$ (mod n)=2^3 (mod 33)= 8 ≡ 8 (mod 33) => ‘I’

por lo que lola recupera el mensaje SI

Para comprobar la firma (firma=24) usando la clave publica de Paco(e=11,n=35):

M=firma^e mod (n)= 24^{11} mod (35)= 19 =>’S’

por lo que se verifica la firma de paco

paco descifra el criptograma (27,15) con su clave privada

$M_1=C_1^d$ (mod n)=27^{11} (mod 35)= 13 => ‘N’

$M_2=C_2^d$ (mod n)=15^{11} (mod 35)=15 => ‘O’

por lo que lola recupera el mensaje NO

Para comprobar la firma (firma=19) usando la clave publica de Lola (e=7,n=33):

M=firma^e mod (n)= 19^7 mod (33)= 13 =>’N’

por lo que se verifica la firma de Lola

13. Si la clave pública del receptor de un criptosistema RSA es el par (n, e)=(2291; 17), el mensaje capturado por el intruso es: 575, encontrar la clave para descifrar el mensaje, par (n,d)=(2291, d), descifrar el mensaje. Indique a qué letra del ASCII corresponde el valor encontrado. Pista: busque en internet el algoritmo de Fermat para encontrar los factores

primos p y q, esto es n=p*q. Codifique en un lenguaje de programación a su elección, el algoritmo de Fermat.

Clave pública del receptor: $(n, e) = (2291, 17)$

Criptograma capturado: $c = 575$

Factorización de $n = 2291$ mediante el método de Fermat

Calculamos a_0 = $\lceil\sqrt{2291}\rceil$ = 48:

- a = 48 => b^2 = 48^2 - 2291 = 2304 - 2291 = 13 (no es cuadrado perfecto)
- a = 49 => b^2 = 49^2 - 2291 = 2401 - 2291 = 110 (no es cuadrado perfecto)
- a = 50 => b^2 = 50^2 - 2291 = 2500 - 2291 = 209 (no es cuadrado perfecto)
- a = 51 => b^2 = 51^2 - 2291 = 2601 - 2291 = 310 (no es cuadrado perfecto)
- a = 52 => b^2 = 52^2 - 2291 = 2704 - 2291 = 413 (no es cuadrado perfecto)
- a = 53 => b^2 = 53^2 - 2291 = 2809 - 2291 = 518 (no es cuadrado perfecto)
- a = 54 => b^2 = 54^2 - 2291 = 2916 - 2291 = 625 = 25^2

Obtenemos $a = 54$ y $b = 25$:

- $p = a - b = 54 - 25 = 29$
- $q = a + b = 54 + 25 = 79$ *(Verificación:* $29 \times 79 = 2291$*)*

Cálculo de $\phi(n)$ y la clave privada d

- $\phi(n) = (p - 1)(q - 1) = (29 - 1)(79 - 1) = 28 \times 78 = 2184$
- Calculamos d como la inversa multiplicativa de $e = 17$ módulo

  $17 \cdot d \equiv 1\ (mod\ 2184)$

  Aplicando el Algoritmo Extendido de Euclides:

  $2184 = 128 \times 17 + 8$

- $17 = 2 \times 8 + 1 \Rightarrow 1 = 17 - 2 \times (2184 - 128 \times 17) = 257 \times 17 - 2 \times 2184$

  Por lo tanto, $d = 257$.

- La clave para descifrar es el par $(n, d) = (2291, 257)$.

Descifrado del mensaje $c = 575$

Aplicamos la fórmula de descifrado RSA $m = c^d\ (mod\ n)$:

$m = 575^{257} \qquad (mod\ 2291) = 65$

- Clave privada d: 257 (Par de descifrado: $(2291, 257)$)
- Mensaje descifrado m: 65

- Carácter ASCII: El entero 65 en la tabla ASCII corresponde a la letra A.

Script:

```
import math

def fermat_factorization(n):
    """
    Factoriza un entero impar n mediante el método de Fermat (n = a^2 - b^2).
    Retorna los factores p y q (n = p * q).
    """
    a = math.isqrt(n)
    if a * a < n:
        a += 1
    while True:
        b2 = a * a - n
        b = math.isqrt(b2)
        if b * b == b2:
            p = a - b
            q = a + b
            return p, q
        a += 1


def resolver_rsa(n, e, c=None):
    """
    Rompe la clave pública RSA (n, e) encontrando p, q, phi(n) y la clave privada d.
    Si se provee un mensaje cifrado c, también lo descifra.
    """
```

```
    p, q = fermat_factorization(n)
    phi = (p - 1) * (q - 1)
    d = pow(e, -1, phi)  # Inverso multiplicativo de e módulo phi(n)

    resultado = {
        "p": p,
        "q": q,
        "phi": phi,
        "d": d
    }

    if c is not None:
        m = pow(c, d, n)
        resultado["m"] = m
        resultado["ascii"] = chr(m) if 0 <= m <= 255 else None

    return resultado

# --- EJERCICIO 13 ---
res13 = resolver_rsa(n=2291, e=17, c=575)
print("=== EJERCICIO 13 ===")
print(f"Factores primos: p = {res13['p']}, q = {res13['q']}")
print(f"Función de Euler phi(n): {res13['phi']}")
print(f"Clave privada d: {res13['d']}")
print(f"Mensaje descifrado (m): {res13['m']}")
print(f"Letra ASCII correspondiente: '{res13['ascii']}'\n")
```

```
# --- EJERCICIO 14 ---

res14 = resolver_rsa(n=536813567, e=3602561)

print("=== EJERCICIO 14 ===")

print(f"Factores primos: p = {res14['p']}, q = {res14['q']}")

print(f"Función de Euler phi(n): {res14['phi']}")

print(f"Clave privada de descifrado d: {res14['d']}")
```

14. Intente romper el criptosistema RSA con clave pública (n, e) = (536813567; 3602561), con el software que escribió en el ejercicio anterior. Para romperlo, encuentre la clave d, que permita descifrar todo mensaje cifrado con la clave pública.

Clave pública: $(n, e) = (536813567, 3602561)$

Factorización de $n = 536813567$ mediante el método de Fermat

Calculamos a_0 = $\lceil\sqrt{536813567}\rceil$ = 23170. Tras iterar sumando 1 a a con el código implementado (13.695 iteraciones):

- $a = 36864$
- $b^2 = 36864^2 - 536813567 = 1358954496 - 536813567 = 822140929 = 28673^2$
- $b = 28673$

Obtenemos los factores primos de n:

- $p = a - b = 36864 - 28673 = 8191$
- $q = a + b = 36864 + 28673 = 65537$ *(Verificación:* $8191 \times 65537 = 536813567$*)*

Cálculo de $\phi(n)$ y obtención de d

$\phi(n) = (p - 1)(q - 1) = (8191 - 1)(65537 - 1) = 8190 \times 65536 = 536739840$

Buscamos d tal que $e \cdot d \equiv 1\ (mod\ \phi(n))$:

$3602561 \cdot d \equiv 1 \qquad (mod\ 536739840)$

Obtenemos el inverso multiplicativo mediante el algoritmo extendido de Euclides:
$d = 201934721$

(Verificación: (3602561 X 201934721) (mod 536739840) = 1*)*

Clave privada que rompe el sistema: $d = 201934721$ (Par de descifrado completo: $(536813567, 201934721)$), permitiendo descifrar cualquier mensaje cifrado con dicha clave pública.

15. INVESTIGACIÓN:

a. Investigue algún algoritmo para el cálculo rápido de potencias, para usar en expresiones xm mod(n).

b. Investigue algún algoritmo para factorizar un número natural N.

a. Algoritmos para el cálculo rápido de potencias: $x^m \bmod n$

El cálculo ingenuo de x^m mediante multiplicaciones sucesivas requiere $m - 1$ multiplicaciones, para $m \geq 1$, por lo que su complejidad es $O(m)$.

Para exponentes de cientos o miles de bits, m es extremadamente grande y este procedimiento resulta inviable en tiempo. Si además se calcula la potencia completa antes de reducirla módulo n, se necesitan números intermedios de gran tamaño.

Para resolverlo, se utiliza la exponenciación modular rápida, aprovechando la propiedad:

$(a \cdot b) \bmod n = ((a \bmod n) \cdot (b \bmod n)) \bmod n$

Algoritmo de exponenciación binaria (square-and-multiply). La exponenciación binaria, de derecha a izquierda o de izquierda a derecha, reduce la cantidad de multiplicaciones de $O(m)$ a $O(\log_2 m)$.

Fundamento:

Todo exponente entero positivo m puede expresarse en binario:

$m = (b\square\ b\square_{-1} \ldots b_1\ b_0)_2$

$= \Sigma_{i=0}^{k}\ b_i \cdot 2^i$

$b_i \in \{0, 1\}$

Por lo tanto:

$x^m = x\text{^}(\Sigma_{i=0}^{k}\ b_i \cdot 2^i)$

$= \prod_{i=0}^{k} (x\text{^}(2^i))\text{^}b_i$

Se calculan potencias mediante cuadrados sucesivos y se multiplican únicamente aquellas cuyos bits correspondientes son iguales a 1.

Algoritmo de derecha a izquierda

Precondiciones: exponente entero ≥ 0 y módulo entero > 0.

```
def modular_pow(base, exponente, modulo):
    resultado = 1 % modulo
    base = base % modulo
    while exponente > 0:
        # Si el bit actual es 1, incorporar la potencia.
        if exponente % 2 == 1:
            resultado = (resultado * base) % modulo
        # Elevar la base al cuadrado para el siguiente bit.
        base = (base * base) % modulo
        # Desplazar el exponente un bit a la derecha.
        exponente //= 2
    return resultado
```

Ejemplo numérico

Calcular: $7^{11} \bmod 13$

El exponente en binario es: $11 = (1011)_2$

Los bits se procesan de derecha a izquierda: 1, 1, 0, 1.

- Iteración 1: bit = 1.
  resultado = $(1 \times 7) \bmod 13 = 7$
  base = $7^2 \bmod 13 = 10$
- Iteración 2: bit = 1.
  resultado = $(7 \times 10) \bmod 13 = 5$
  base = $10^2 \bmod 13 = 9$
- Iteración 3: bit = 0.
  resultado se mantiene en 5.
  base = $9^2 \bmod 13 = 3$

• Iteración 4: bit = 1.

resultado = (5 × 3) mod 13 = 2

El exponente pasa a cero y finaliza el algoritmo.

Resultado:

$7^{11} \equiv 2 \pmod{13}$

Optimización: reducción de montgomery

En implementaciones con enteros grandes, realizar una división para obtener el resto en cada multiplicación puede ser costoso.

La multiplicación de Montgomery utiliza un cambio de representación:

$x \rightarrow x \cdot R \bmod n$

Se elige R de forma que:

$R > n$

$\text{mcd}(R, n) = 1$

Habitualmente, R es una potencia de 2 y n es impar. Esto permite sustituir las divisiones generales por n por operaciones que aprovechan desplazamientos de bits, máscaras y correcciones. Su utilidad aumenta cuando se realizan muchas multiplicaciones con el mismo módulo, porque se amortiza el costo de convertir los operandos.

b. Algoritmos para factorizar un número natural n

La factorización consiste en descomponer un entero N > 1 en factores primos. En RSA, el módulo suele ser el producto de dos primos grandes:

$N = p \cdot q$

La dificultad de factorizar este producto es fundamental para la seguridad de RSA. Otros sistemas asimétricos se basan en problemas diferentes.

Según el tamaño de N y las características de sus factores, se utilizan distintos algoritmos.

1. División de ensayo (trial division)

Descripción: Probar la divisibilidad de N por los números primos p que cumplen:

$p \leq \sqrt{N}$

Todo número compuesto tiene al menos un factor primo menor o igual que su raíz cuadrada.

Complejidad: $O(\sqrt{N})$

Esta es una cota sencilla para la cantidad de pruebas de divisibilidad, sin contar el costo de la aritmética con enteros grandes.

Uso: Adecuada para números pequeños o como paso previo para eliminar factores primos pequeños, como 2, 3, 5 y 7.

2. Algoritmo ρ de pollard (pollard's rho)

Fundamento: Utiliza sucesiones de apariencia pseudoaleatoria generadas por una función como:

$f(x) = (x^2 + c) \bmod N$

Su comportamiento se relaciona con la paradoja del cumpleaños: se buscan colisiones módulo un factor primo desconocido de N.

Funcionamiento:

1. Se emplea la detección de ciclos de Floyd, denominada “tortuga y liebre”:

$x_{i+1} = f(x_i)$

$y_{i+1} = f(f(y_i))$

2. En cada paso se calcula:

$d = \text{mcd}(|x_i - y_i|, N)$

3. Se interpreta el resultado:

$d = 1$ → continuar.

$1 < d < N$ → d es un factor no trivial.

$d = N$ → reiniciar con otros parámetros.

Complejidad esperada heurística: $O(\sqrt{p})$

donde p es el menor factor primo de N.

Si $N = p \cdot q$ y ambos factores son de tamaños similares:

$p \approx \sqrt{N}$

Entonces:

$O(\sqrt{p}) \approx O(N^{(1/4)})$

Uso: Es útil para encontrar factores relativamente pequeños dentro de números grandes. No resulta, en general, eficiente para buscar factores primos de 50–60 dígitos decimales.

IMPLEMENTACIÓN DE UN INTENTO

```
import math

def pollard_rho(N):
    if N % 2 == 0:
        return 2
    x, y, d = 2, 2, 1
    f = lambda x: (x**2 + 1) % N
    while d == 1:
        x = f(x)
        y = f(f(y))
        d = math.gcd(abs(x - y), N)
    return d
```

Punto Crítico: Esta implementación didáctica presupone que N es compuesto. Si devuelve N, el intento falló y debe repetirse cambiando la semilla o la constante c. El resultado N no constituye un factor no trivial.

3. Método de factorización de fermat

Fundamento: Busca expresar un número impar N como diferencia de cuadrados:

$$N = a^2 - b^2$$

$$= (a - b)(a + b)$$

Funcionamiento:

Se comienza con:

$$a = \lceil \sqrt{N} \rceil$$

Luego se prueban valores sucesivos de a, comprobando si:

$$a^2 - N = b^2$$

es un cuadrado perfecto.

Si se encuentra, los factores son:

$$p = a - b$$

$$q = a + b$$

Uso: Es especialmente rápido cuando los factores p y q están muy próximos entre sí:

$$p \approx q \approx \sqrt{N}$$

Los factores 2 se extraen previamente. Si los factores restantes están muy separados, el método puede requerir muchas iteraciones.

4. Criba cuadrática (quadratic sieve, qs)

Fundamento:

Busca enteros x e y tales que:

$$x^2 \equiv y^2 \pmod{N}$$

con:

$$x \not\equiv y \pmod{N}$$

$$x \not\equiv -y \pmod{N}$$

Esto permite obtener factores no triviales calculando:

$$\mathrm{mcd}(x - y, N)$$

$$\mathrm{mcd}(x + y, N)$$

Para construir estas congruencias, se recopilan relaciones con factores primos pequeños y se utiliza álgebra lineal sobre GF(2).

Complejidad heurística subexponencial: $\exp((1 + o(1)) \cdot \sqrt{(\ln N \cdot \ln \ln N)})$

Uso: Es un método de propósito general útil para números de tamaño intermedio. El punto en que conviene reemplazarlo por GNFS depende de la implementación y de los recursos disponibles.

5. Criba general del cuerpo de números (gnfs)

Fundamento: Construye relaciones mediante aritmética sobre enteros y cuerpos de números algebraicos. Estas relaciones permiten obtener congruencias de cuadrados y extraer factores mediante el máximo común divisor.

Complejidad heurística subexponencial: $\exp((c + o(1)) \cdot (\ln N)^{(1/3)} \cdot (\ln \ln N)^{(2/3)})$

donde:

$$c = (64/9)^{(1/3)} \approx 1{,}923$$

Uso: Es un algoritmo clásico de referencia para factorizar enteros generales muy grandes, incluidos módulos RSA.

Punto Crítico: El tamaño de una clave RSA no garantiza por sí solo la seguridad. También influyen la generación de los primos, el esquema de relleno, la implementación y la protección de la clave privada.